Assignment 1

Smart Priority Vacuum Cleaner

https://github.com/tejasmali46/AIML_lab

Name	Tejas Vinod Mali
PRN	202401040083
Batch	A1
Division	A
Roll No.	25
Subject	Artificial Intelligence and Machine Learning

Title: Implementation of an Intelligent Vacuum Cleaner Agent

Application: Smart Priority-Based Room Cleaning

1. AIM

To design and implement a smart vacuum-cleaning agent in Python that observes a multi-room environment, selects a room based on dust level and priority, cleans the room, handles obstacles, manages its battery and dust bin, and generates a final performance report.

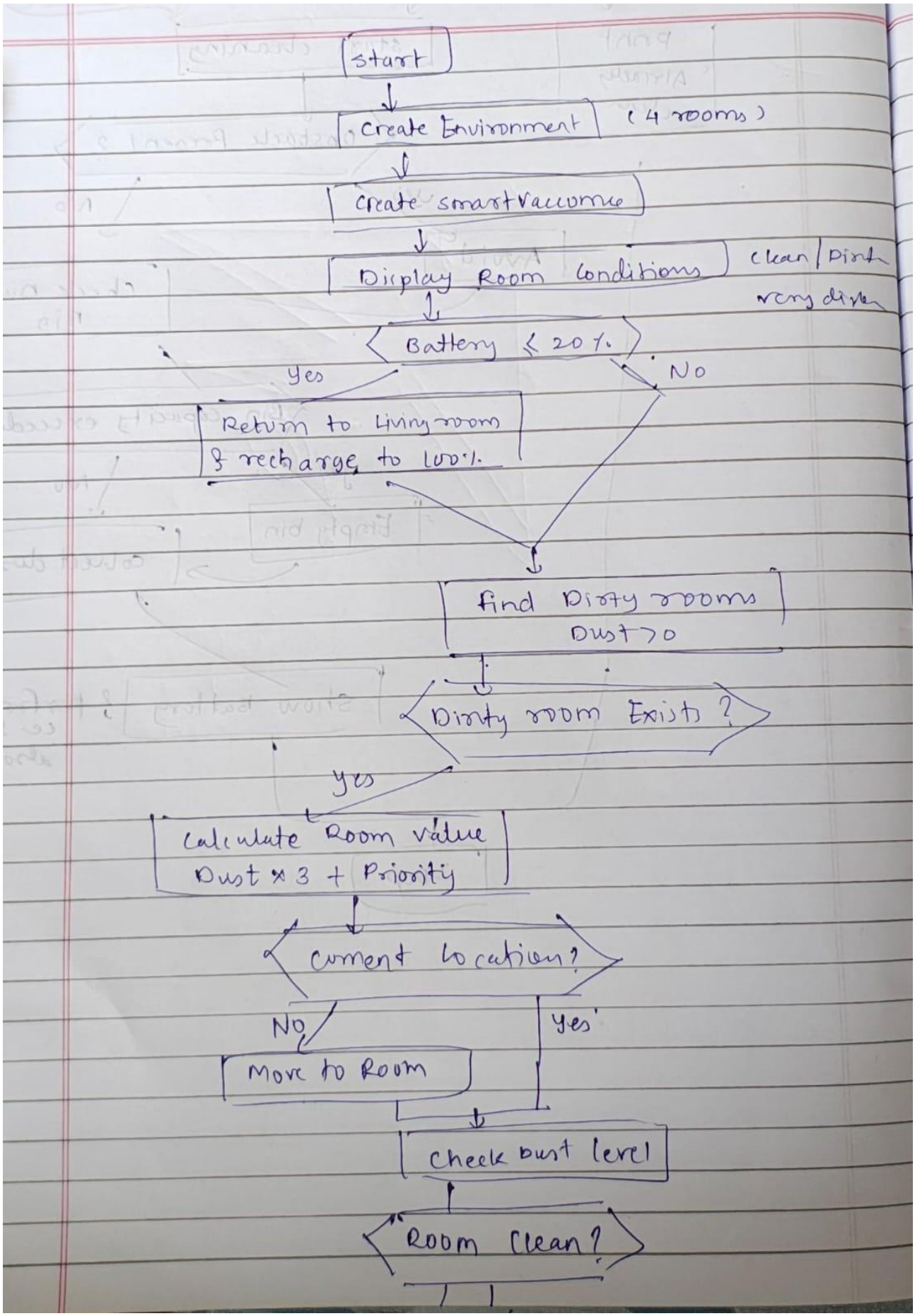
2. THEORY

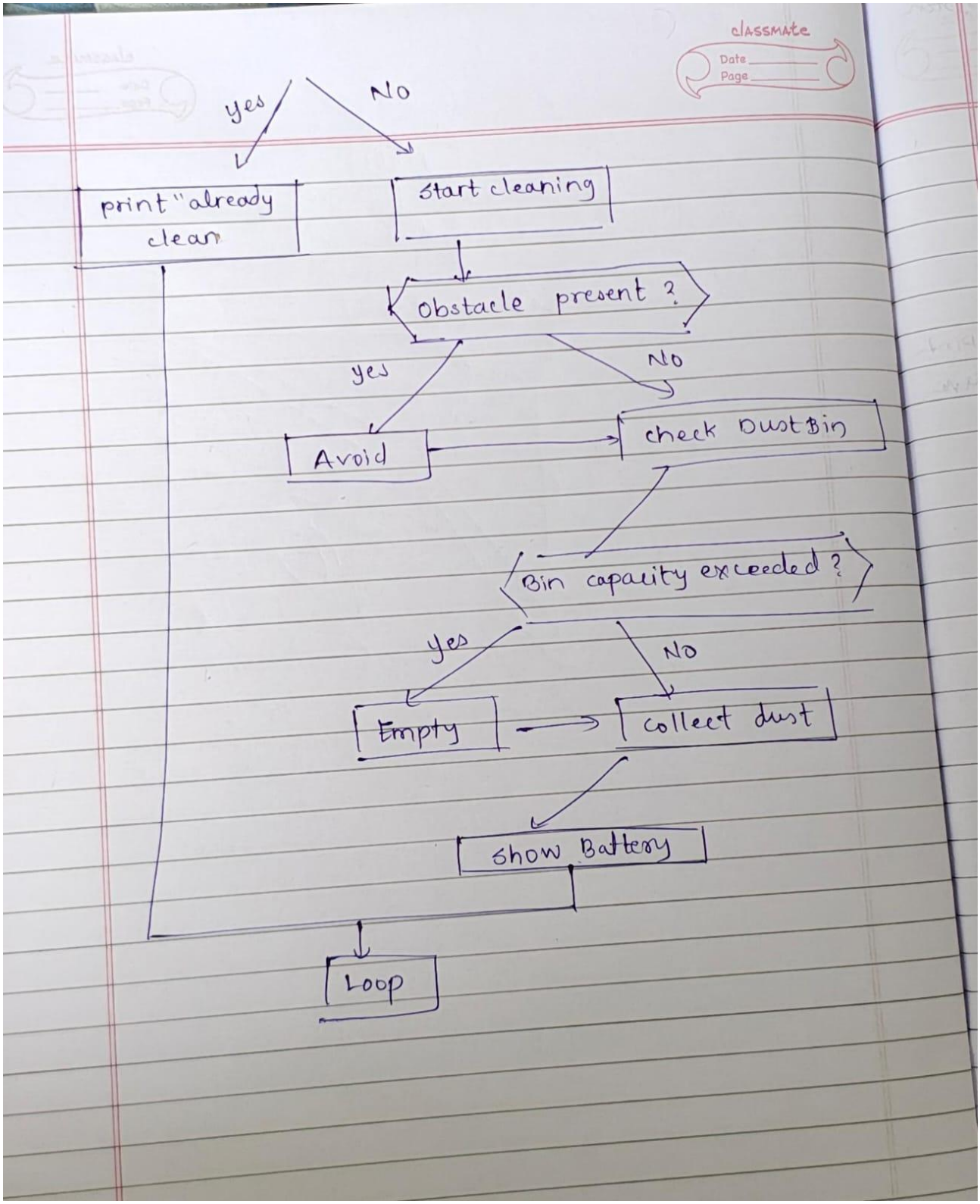
- **Intelligent Agent:** An agent observes its environment, makes decisions and performs actions to achieve a goal.
- **Environment:** The simulated house contains Living Room, Kitchen, Bedroom and Balcony.
- **Dust Level:** 0 = Clean, 1 = Dusty, 2 = Very Dirty.
- **Priority:** Each room has a fixed importance. The agent calculates Value = (Dust × 3) + Priority.
- **Battery:** Movement, cleaning and obstacle handling consume battery. At 20% or below, the agent recharges.
- **Dust Bin:** The bin can hold 4 units. If capacity would be exceeded, it is emptied before further collection.
- **Performance:** Rooms cleaned, movements, battery, dust-bin level and score are recorded.

3. PEAS

Component	Description
Performance	Rooms cleaned, dust removed, movements, battery usage and performance score.
Environment	Four rooms with dust, room priority and possible obstacles.
Actuators	Movement, cleaning, obstacle handling, recharge and dust-bin emptying.
Sensors / Percepts	Room location, dust level, obstacle status, battery and dust-bin state.

4. FLOWCHART


Start
Create Environment
(4 rooms)
Create smartVaccumce
Display Room Conditions
Battery < 20%
Yes
No
Return to Living room
& recharge to 100%.
find Dirty rooms
Dust > 0
Dirty room Exists ?
yes
Calculate Room value
Dust * 3 + Priority
current location?
No
Yes
Move to Room
Check Dust level
Room Clean?


classmate
Date
Page
yes
No
print "already clean
Start cleaning
Obstacle present ?
yes
No
Avoid
check DustBin
Bin capacity exceeded ?
yes
No
Empty
collect dust
show Battery
Loop

4. OBSERVATION

Because dust levels, obstacles and starting location are random, the exact output changes at every run. The following table shows one possible observation.

Parameter	Example Observation
Starting location	Kitchen
Living Room	Dusty • Priority 2 • No obstacle
Kitchen	Very Dirty • Priority 3 • Obstacle present
Bedroom	Clean • Priority 1 • No obstacle
Balcony	Dusty • Priority 2 • No obstacle
Dust-bin capacity	4 units
Recharge condition	Battery □ 20%

Expected observations:

- Robot starts from a random room.
- Dirty rooms are compared using dust level and room priority.
- The highest-value room is selected for cleaning.
- Obstacles are handled before cleaning.
- The dust bin is emptied when necessary.
- The robot recharges when battery reaches 20% or below.
- The robot stops automatically after all rooms are clean.

5. CODE

```
import random


class Environment:

    def __init__(self):
        self.rooms = {
            "Living Room": {
                "dust": random.randint(0, 2),
                "priority": 2,
                "obstacle": random.choice([True, False])
            },
            "Kitchen": {
                "dust": random.randint(0, 2),
                "priority": 3,
                "obstacle": random.choice([True, False])
            },
            "Bedroom": {
                "dust": random.randint(0, 2),
                "priority": 1,
                "obstacle": random.choice([True, False])
            },
            "Balcony": {
                "dust": random.randint(0, 2),
                "priority": 2,
                "obstacle": random.choice([True, False])
            }
        }

    def display(self):
```

```
        print("\nROOM CONDITIONS")

        for room, data in self.rooms.items():
            dust = data["dust"]

            #checking room condition
            if dust == 0:
                status = "Clean"
            elif dust == 1:
                status = "Dusty"
            else:
                status = "Very Dirty"

            obstacle = "Yes" if data["obstacle"] else "No"
            print(room, "| Dust:", status, "| Priority:", data["priority"], "| Obstacle:", obstacle)

class SmartVacuum:

    def __init__(self, environment):
        self.env = environment

        #starting from a random room
        self.location = random.choice(list(self.env.rooms.keys()))

        self.battery = 100
        self.dust_bin = 0
        self.bin_capacity = 4
        self.score = 0
        self.rooms_cleaned = 0
        self.movements = 0
```

```
65.
66.     def choose_room(self):
67.         dirty_rooms = {}
68.
69.         for room, data in self.env.rooms.items():
70.
71.             #finding dirty rooms
72.             if data["dust"] > 0:
73.                 value = data["dust"] * 3 + data["priority"]
74.                 dirty_rooms[room] = value
75.
76.         #all rooms are clean
77.         if not dirty_rooms:
78.             return None
79.
80.         #choosing the room with more dust
81.         return max(dirty_rooms, key=dirty_rooms.get)
82.
83.     def clean(self):
84.         data = self.env.rooms[self.location]
85.         dust = data["dust"]
86.
87.         #room is already clean
88.         if dust == 0:
89.             print(self.location, "is already clean.")
90.             return
91.
92.         print("\nCleaning", self.location)
93.
94.         #checking for obstacle
```

```
        if data["obstacle"]:
            print("Obstacle found!")
            print("Robot is moving around it...")
            self.battery -= 5

            #obstacle is cleared
            data["obstacle"] = False

        #checking dust bin
        if self.dust_bin + dust > self.bin_capacity:
            print("Dust bin is full.")

            #emptying the dust bin
            self.empty_bin()

        #collecting dust
        self.dust_bin += dust

        #making the room clean
        data["dust"] = 0

        #battery used for cleaning
        self.battery -= dust * 10

        #adding cleaning score
        self.score += dust * 10
        self.rooms_cleaned += 1

        print(" ", self.location, "cleaned successfully.")
        print("Dust collected:", self.dust_bin, "/", self.bin_capacity)
```

```
125.
126.     def move(self, destination):
127.
128.         #no need to move
129.         if self.location == destination:
130.             return
131.
132.         #moving to next room
133.         print("\nMoving from", self.location, "to", destination)
134.
135.         self.location = destination
136.         self.battery -= 5
137.         self.movements += 1
138.
139.         print("Robot reached", destination)
140.
141.     def recharge(self):
142.
143.         print("\nBattery is low.")
144.
145.         #going back for charging
146.         print("Returning to charging station...")
147.
148.         self.location = "Living Room"
149.         self.battery = 100
150.
151.         print("Battery fully charged.")
152.
153.     def empty_bin(self):
154.
```

```
155.         print("\nDust bin is full.")
156.
157.         #going to empty the bin
158.         print("Returning to Living Room...")
159.
160.         self.location = "Living Room"
161.         self.dust_bin = 0
162.
163.         print("Dust bin emptied.")
164.
165.     def check_battery(self):
166.
167.         #battery is getting low
168.         if self.battery <= 20:
169.             self.recharge()
170.
171.     def run(self):
172.
173.         print("\nSMART PRIORITY VACUUM ACTIVATED")
174.         print("Starting Location:", self.location)
175.
176.         self.env.display()
177.
178.         while True:
179.
180.             #checking battery
181.             self.check_battery()
182.
183.             #finding the next room
184.             next_room = self.choose_room()
```

```
185.
186.            #all rooms are clean
187.            if next_room is None:
188.                print("\n✓ All rooms are clean!")
189.                break
190.
191.            #moving to next room
192.            self.move(next_room)
193.
194.            #cleaning the room
195.            self.clean()
196.
197.            print("\nBattery:", self.battery, "%")
198.            print("Performance Score:", self.score)
199.
200.        self.final_report()
201.
202.    def final_report(self):
203.
204.        print("\nFINAL REPORT")
205.        print("Rooms Cleaned:", self.rooms_cleaned)
206.        print("Movements:", self.movements)
207.        print("Dust Bin:", self.dust_bin, "/", self.bin_capacity)
208.        print("Final Battery:", self.battery, "%")
209.        print("Performance:", self.score)
210.
211.        self.env.display()
212.
213. #creating environment
214. environment = Environment()
```

```
215.
216. #creating vacuum
217. vacuum = SmartVacuum(environment)
218.
219. #starting the vacuum
220. vacuum.run(
```

6. RESULT ANALYSIS

- The program creates a dynamic multi-room environment using random values.
- The agent does not follow a fixed room order; it selects a room using dust and priority.
- Battery consumption makes the agent monitor its internal state.
- Obstacle handling changes the action taken by the agent.
- Dust-bin capacity introduces another decision condition.
- The score and movement count provide measurable performance information.
- The program stops only when no dirty rooms remain.

7. RESULT

The Smart Priority Vacuum Cleaner was successfully implemented in Python. It can select and clean rooms, handle obstacles, manage battery and dust-bin conditions, and produce a final performance report.

8. CONCLUSION

The experiment demonstrates the basic working of an intelligent agent. The vacuum observes the simulated environment, makes a decision using room condition and priority, performs an action, updates its state and repeats the process until the cleaning goal is achieved. The additional battery, obstacle, dust-bin and performance features make the agent more realistic than a simple fixed two-room vacuum cleaner.